

Hirugarren Entrega

Kudeaketaren eta Informazio Sistemen Informatikaren Ingenieritzako Gradua

Informazio Sistemen Analisia eta Diseinua

Ahultasunen konponketa

Urko Bidaurre

Ander Ibarra

Yassin Mehani

Eneko Puente

Unai Rodriguez

Asier Sinobas

Irakaslea

Mikel Egaña Aranguren

2025eko azaroaren 14a

Gaien aurkibidea

1 Sarrera	3
1.1 Deskribapena	3
2 Konponketak	4
2.1 Anti-CSRF tokenen falta	4
2.2 SQL Injekzioa	6
2.3 Zerbitzariak bertsio-informazioa iragazten du http erantzun-goiburuko "Server" eremuaren bidez	7
2.4 CSP eta Anti-Clickjacking goiburuen falta	8
2.5 Zerbitzariak HTTP "X-Powered-By" erantzun-goiburuko eremu(ar)en bidez informazioa zabaltzen du	10
2.6 Cookie HttpOnly eta SameSite parametrorik gabe	11

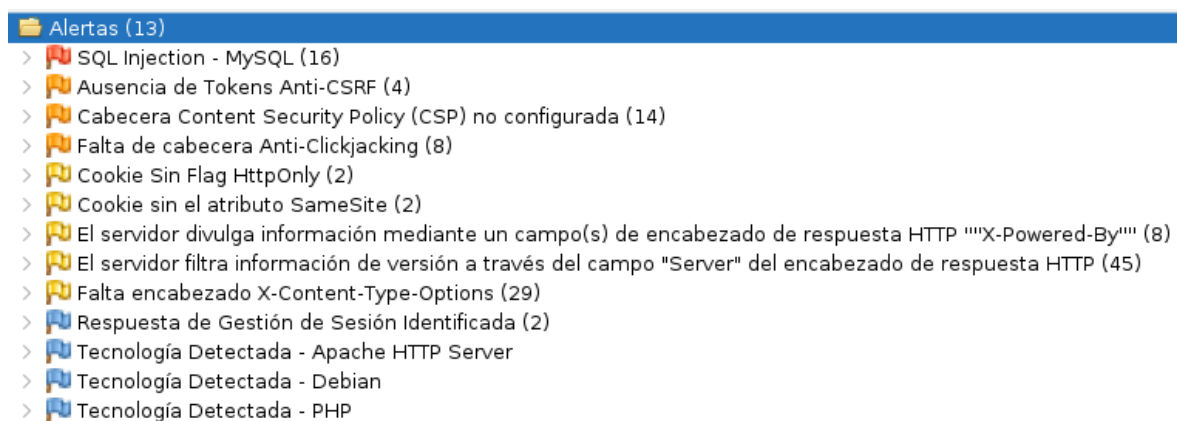
1 Sarrera

Txosten honen helburua gure proiektuaren bigarren entregan aurkitutako segurtasun ahuleziak konpontzeko emandako pausuen azalpen zehatzak ematea da.

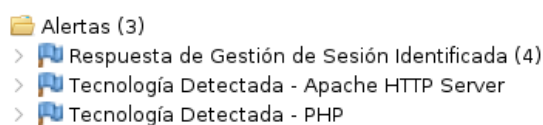
1.1 Deskribapena

Ahuleziak detektatzeko eta horien zuzenketa berresteko, [ZAP](#) pentesting tresna erabili dugu, eta honen analisiaren emaitza oinarritu dugu txosten hau.

Hona hemen ZAP-en analisiaren emaitza, zuzenketak aplikatu baino lehen:



Eta hona hemen ZAP-en analisiaren emaitza, ondoren azalduko diren zuzenketak aplikatu ostean:



Gainerako hiru alerta hauek informazioa baino ez dira eta ez dira segurtasun arazoak.

2 Konponketak

Atal honetan, aurretik ikusitako erroreak nola zuzendu diren jorratuko da. Ez dira arazo bakoitzerako *commit*-en erreferentziak adierazi, izan ere, *commit* bakoitzak arazo bat baino gehiagotan eragina duten aldaketak biltzen ditu, eta arazo bakoitzerako egindako modifikazioak ulertzea oztopatzen du. Horren ordez, kodean egin diren aldaketa guztiak zehatz-mehatz azaldu dira, bakoitzeko kode puskak adieraziz.

2.1 Anti-CSRF tokenen falta

CSRF (Cross-Site Request Forgery) erasoak gerta daitezke erasotzaileak baimendutako erabiltzaile legitimo baten saioa erabiltzen duenean eskaera ez-legitimoak egiteko. Helmugako webguneak edo zerbitzariak ez badu erabiltzailearen identitatea egiaztatzen eskaera bat prozesatzean, segurtasun arazo honen arriskuan egongo da. Ondorioz, erasoak jaso duen erabiltzaileak web-sisteman egin dezakeen edozein ekintza burutzeko gai izango dira erasotzaileak.

Segurtasun arazo larri hau ekiditeko, Anti-CSRF tokenen sorkuntza inplementatu dugu gure web aplikazioan. Horretarako [csrf.php](#) fitxategia sortu dugu, saio cookie-ak ([ikusi 2.6 atala](#)) eta Anti-CSRF tokenak sortuko dituenak. Anti-CSRF token bat emanda, token hori baliozkoa den ala ez adierazteko funtzio bat sortu dugu ere. Token hori saioaren hasieran sortuko da, eta datu-baseari eskaerak egitean tokenak balioztatuko dira.

Saioa hasteko pantaila adibide moduan hartuta, hurrengo sekuentziaren bitartez egiaztatzen da orain saioa baliozkoa dela:

1. Orria kargatzean, JavaScript-etik Anti-CSRF tokena eskatzen zaio aurretik aipatutako *csrf.php*-ri.

```
try {
  // CSRF tokena eskuratu
  const response = await fetch("../config/csrf.php");
  const data = await response.json();

  if (data.csrf_token) {
    csrfInput.value = data.csrf_token; // CSRF tokena formularioan ezarri
  } else {
    console.error("CSRF tokena eskuratzea ezinezkoa izan da.");
  }
} catch (error) {
  console.error("Errorea CSRF tokena eskuratzean:", error);
}
```

Tokena zuzen jaso bada, HTML-an dagoen ezkututako elementuan gehituko da, geroago formulario osoarekin bidaltzeko.

```
<form name="flogin" id="login_form" action="../config/login.php" method="POST">
  <h1 style="text-align: center;">SAIOA HASI</h1>
  <input type="hidden" name="csrf_token">
```

2. Saioa hasteko botoia (*login_submit*) sakatzean formularioaren datuak bidaliko dira *login.php*-ra ohiko moduan, baina orain haien artean Anti-CSRF tokena izango dugu.
3. Eskaera jasotzean, *login.php* fitxategiak bidalitako tokena egiaztatuko du, *csrf.php*-ren *validateCsrfToken* funtzioaren bitartez. Tokena zuzena bada aurrera egingo du, eta ez bada baliozkoa errorea itzuliko du.

```
include_once __DIR__ . '/csrf.php';
// CSRF tokena egiaztatu
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (!validateCsrfToken($_POST['csrf_token'] ?? '')) {
        echo json_encode([
            "success" => false,
            "message" => "CSRF tokena baliogabea."
        ]);
        exit;
    }
}
```

4. Tokena zuzena izan bada aurreko pausuan, datu-basean kontsulta egingo da, beti bezala.

Aldaketa hauek formulario bat zuten orri guztietan inplementatu dira (*login*, *register*, *add_item*, *modify_item* eta *modify_user*).

2.2 SQL Injekzioa

SQL injekzioaren bidez, erasotzaile batek datu-basea era arriskutsu batean manipula dezake. Erasotzaileak kontsulta bat egitean eduki arbitrarioa txerta dezake datu-basean (kontsulta manipulatzuz, ustekabeko balioak gehituz edo datu kaltegarriak sartuz), SQL gehigarria injektatu (eremu berriak sortu, datuak gainidatzi edo kontsulta hautsi beste bat exekutatzeko), edo beste taula batzuetan idatzi (sintaxia apurtuz, VALUES itxiz eta bere jarraibideekin jarraituz), eta abar.

Hau ekiditeko, parametroak zuzenean hartzen zituzten kontsultak, *PreparedStatement* baten ordez aldatu ditugu:

1. Kartak gehitzerako orduan.

Bertsio zaharra:

```
// SQL kontsulta zuzenean parametroekin
$sql = "INSERT INTO Datuak (izena, kostua, bizitza, eraso, mota)
      VALUES ('$item_izena', '$item_kostua', '$item_bizitza', '$item_eraso', '$item_mota')";
```

Bertsio berria:

```
// Prepared statement bat SQL injekzioak saihesteko
$stmt = $conn->prepare("INSERT INTO Datuak (izena, kostua, bizitza, eraso, mota) VALUES (?, ?, ?, ?, ?)");
if ($stmt === false) {
    echo json_encode(['success' => false, 'message' => 'Errorea kontsulta prestatzean: ' . $conn->error]);
    $conn->close();
    exit;
}

// Parametroak string gisa lotu, injekzio-arriskuak saihesteko.
if (!$stmt->bind_param('siiiis', $item_izena, $item_kostua, $item_bizitza, $item_eraso, $item_mota)) {
    echo json_encode(['success' => false, 'message' => 'Parametroak lotzean errorea: ' . $stmt->error]);
    $stmt->close();
    $conn->close();
    exit;
}
```

2. Erabiltzaileak erregistratzean:

Bertsio zaharra:

```
// Erabiltzailea datu-basean sartu
$user_insert = mysqli_query($conn, "INSERT INTO Erabiltzailea (nan, izena, jaiotze_data, tlf, email, pasahitza)
      VALUES ('$nan', '$izena', '$data', '$zenbaki', '$email', '$pasahitza')");
```

Bertsio berria:

```
// Erabiltzailea datu-basean sartu
$stmt = mysqli_prepare($conn, "INSERT INTO Erabiltzailea (nan, izena, jaiotze_data, tlf, email, pasahitza) VALUES (?, ?, ?, ?, ?, ?)");
mysqli_stmt_bind_param($stmt, 'ssssss', $nan, $izena, $data, $zenbaki, $email, $pasahitza);
$user_insert = mysqli_stmt_execute($stmt);
```

3. Erabiltzaileen datuak eguneratzeko:

Bertsio zaharra:

```
// Erabiltzailea eguneratzeko SQL kontsulta
$sql = "UPDATE Erabiltzailea
      SET izena='{$_POST['izena']}', email='{$_POST['email']}', jaiotze_data='{$_POST['dob']}', tlf='{$_POST['telefonoa']}'
      WHERE nan='{$_POST['nan']}';
```

Bertsio berria:

```
// Erabiltzailea eguneratzeko SQL kontsulta
$stmt = mysqli_prepare($conn, "UPDATE Erabiltzailea SET izena=?, email=?, jaiotze_data=?, tlf=? WHERE nan=?");
mysqli_stmt_bind_param($stmt, 'ssssss', $_POST['izena'], $_POST['email'], $_POST['dob'], $_POST['telefonoa'], $_POST['nan']);
$sql = mysqli_stmt_execute($stmt);
```

PreparedStatement erabiltzeari esker, sarrera guztiak datu bezala tratatzen dira, ez SQL kode bezala, hau da, kontsulta eta datuak banatuta mantentzen dira, eta ezin izango da kontsultaren sintaxia hautsi edo manipulatu.

2.3 Zerbitzariak bertsio-informazioa iragazten du *http* erantzun-goiburuko “Server” eremuaren bidez

Web orrialdearen segurtasuna hobetzeko bertsio-informazioa eskuragarri ez egotea garrantzitsua da. Horretarako, Dockerfile fitxategian aldaketak egin ditugu, hurrengo kodea gehituz:

```
RUN    printf "ServerTokens Prod\nServerSignature Off\n" >
/etc/apache2/conf-available/security.conf \&& printf
"<IfModule mod_headers.c>\n
    Header always unset Server\n
</IfModule>\n" >> /etc/apache2/conf-available/security.conf \
&& a2enconf security
```

Gehikuntza honen bitartez, *security.conf*-en barruan **ServerTokens Prod** eta **ServerSignature Off** idazten da. Lehenengo lerroak server goiburuan Apache-ren bertsioa agertzen ez dela ziurtatzen du, eta bigarrenak Apache-k sortutako errore-orrien oinetan agertzen den sinadura ezabatzen du.

Horrez gain, “**Header always unset Server**” lerroak server goiburua ezabatzen du erantzun guztietatik. Eta azkenik, “*a2enconf security*” lerroak konfigurazio berri hau aplikatzen du.

2.4 CSP eta Anti-Clickjacking goiburuen falta

X-Frame-Options/frame-ancestors-ek eta CSP-k bi mehatxu desberdin estaltzen dituzte (*clickjacking*-a eta XSS/baimendu gabeko baliabideak), eta web sistema batean ezinbestekoa da biak aplikatzea, formularioen fluxuak babesteko eta defentsa sakona bermatzeko, nabigatzaile zaharrekiko bateragarritasuna emateaz gain.

Honen harira, bi goiburu hauek aplikatzeko, hurrengo aldaketak egin ditugu gure web sisteman:

1. Apache-k fitxategi estatiko eta dinamikoetarako goiburuak ezar ditzan gaitzea (*mod_headers* eta *AllowOverride* aktibatzea). Hau lortzeko, Dockerfile fitxategian Apache-ren *headers* moduloa gaitu dugu eta konfigurazio fitxategi berri bat sortu dugu *conf-available* direktorioan, *AllowOverride All* zehazten duena:

```
# Instalatu mysql-i hedadura eta prestatu apache moduluak
RUN docker-php-ext-install mysqli \
    && a2enmod headers

# Ziurtatu .htaccess fitxategiak errespetatzen direla web erroaren gainidazpenak
baimenduz.
# Conf fitxategi txiki bat gehitu eta gaitu.
RUN printf "<Directory /var/www/html/>\n    AllowOverride All\n</Directory>\n" >
/etc/apache2/conf-available/allow-override.conf \
    && a2enconf allow-override
```

2. Web-zerbitzuaren definizioa aldatu *build: .* erabil dezan, eta ez *image: web*, Docker-rek aurreko doikuntzak dituen *Dockerfile*-a erabiliz irudia eraiki dezan.

```
services:
  web:
    build: .
    environment:
      - ALLOW_OVERRIDE=true
    ports:
      - "81:80"
    links:
      - db
    volumes:
      - ./app:/var/www/html/
  ...
```


3. Segurtasun-goiburuak `.htaccess`-en erantsi (direktiba honek fitxategi estatikoak estaltzen dituelako: HTML, CSS eta JS)

```
DirectoryIndex index.html index.php

<IfModule mod_headers.c>
    Header always set X-Frame-Options "DENY"
    Header always set X-Content-Type-Options "nosniff"
    Header always set Referrer-Policy "no-referrer-when-downgrade"
    Header always set Content-Security-Policy "default-src 'self';script-src
'self';style-src 'self';img-src 'self' data:;frame-ancestors 'none';form-action
'self';base-uri 'self'"
</IfModule>
```

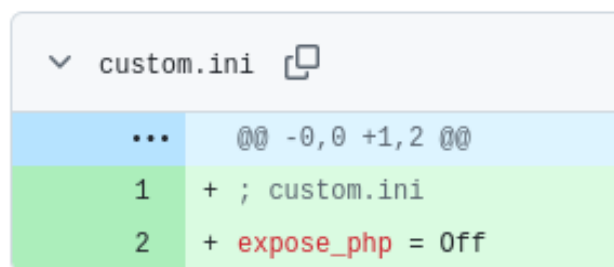
- **Header always set X-Frame-Options "DENY":** *framing*-a saihesten du (*anti-clickjacking*).
- **X-Content-Type-Options** eta **Referrer-Policy:** segurtasuna indartzen dute.
- **CSP goiburua:**
 - **default-src 'self':** berariazko zuzentzaurik ez duten baliabide mota guztietarako politika lehenetsia ezartzen du. 'self' hitzak "jatorri berekoa bakarrik" esan nahi du (protokoloa+host+portua).
 - **script-src 'self':** script-ak (JavaScript) non kargatu eta exekutatu daitezkeen kontrolatzen du. 'self' -ek gure domeinuak zerbitzatutako script-ak bakarrik ahalbidetzen ditu.
 - **style-src 'self':** estilo-orriak kontrolatzen ditu. *script-src*-ren antzekoa.
 - **img-src 'self' data:** irudietarako baimendutako jatorriak kontrolatzen ditu. *data:*-k URI datekin sartutako irudiak ahalbidetzen ditu.
 - **frame-ancestors 'none':** adierazten du zein jatorriek embeditu dezaketen gure orria frame/iframe batean. 'None'-ek erabat debekatzen du embeditua izatea.
 - **form-action 'self':** definitzen du zein jatorritara bidal daitezkeen formularioak gure orritik (formularioen *action* atributua). 'self' -ek mugatu egiten ditu bidalketak gure domeinu propioa.
 - **base-uri 'self':** `<base>` elementuaren balio baimendua mugatzen du (URI erlatiboak nola konpontzen diren aldatzen duena). 'self' -ek `<base>` batek gure basea kanpoko domeinu batera aldatzea saihesten du.
- 4. **Edukiontzia berreraiki,** izan ere, *Dockerfile*-a aldatzeak irudia aldatzen du eta aldaketa hauek irudi berri bat eraikitzen denean eta irudi horretatik edukiontzia abiarazten direnean bakarrik aplikatzen dira.

2.5 Zerbitzariak HTTP "X-Powered-By" erantzun-goiburuko eremu(ar)en bidez informazioa zabaltzen du

Ez da web-orriaren akats kritiko bat, baina bai garrantzitsua dela konpontzea, beste hainbat arazo larriagotan aldatu daitekeen datu-esposizio garrantzitsua baita. Datuen azalpen horrek zerbitzariak erabiltzen duen teknologiari buruzko informazioa ematen du. Horrek web-orriaren bertsioari eta ahuleziei buruzko informazio asko ematen dio erasotzaileari, eraso eraginkorragoa egin ahal izateko.

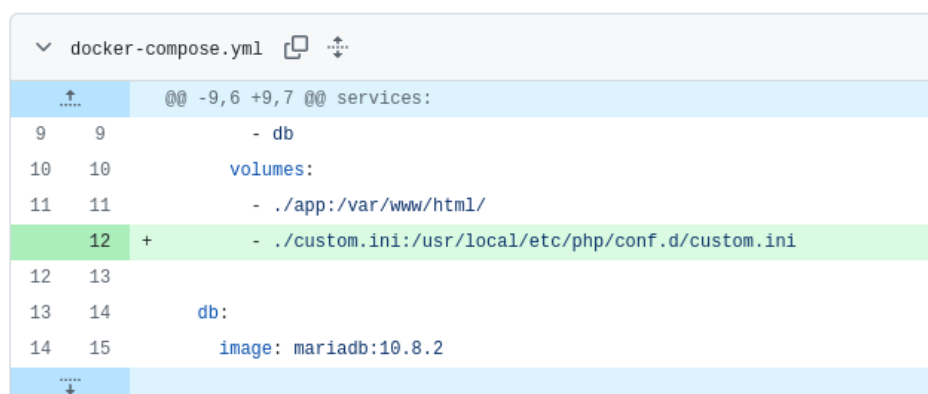
Ahulezi hau konpontzeko aldaketa txiki pare bat egin behar izan ditugu proiektuan:

1. Hasteko, *custom.ini* fitxategia sortu dugu kode-lerro bakarrarekin, non PHPri *X-Powered-By* goiburuan PHPren bertsioa ez erakusteko esaten zaion. Horri esker, zerbitzariari buruz zabaltzen den informazio kopurua murriztu egiten da, gure web-sistemari egin dakizkiokeen erasoak oztopatuz.



```
▼ custom.ini
...
1 + ; custom.ini
2 + expose_php = Off
```

2. Fitxategia sortu ondoren, "*docker-compose up*" komandoa exekutatzen dugun bakoitzean *custom.ini* fitxategia ere exekutatu behar dugu konfigurazio berri hori aplikatzeko. Horretarako, *docker-compose.yml* fitxategia aldatu dugu, *custom.ini* fitxategiaren path-a gehituz "volumes" atalean.



```
▼ docker-compose.yml
↑
@@ -9,6 +9,7 @@ services:
9 9 - db
10 10 volumes:
11 11 - ./app:/var/www/html/
12 12 + - ./custom.ini:/usr/local/etc/php/conf.d/custom.ini
12 13
13 14 db:
14 15 image: mariadb:10.8.2
↓
```

2.6 Cookie HttpOnly eta SameSite parametrarik gabe

Arazo hau hainbat arrazoiengatik gerta daiteke: nabigatzaile modernoak edukitako aldaketengatik, konfigurazio osatugabeengatik, seguntasun-atributurik gabeko cookie-ak sortzeagatik... Gure web gunean, arazoa eman da aurretik zehaztutako cookie-ak sortzen genituelako.

Ondorioz, *HttpOnly* eta *SameSite* parametroak konpondu behar izan ditugu, ahulezi honen presentzia hurrengo arazoak ekar zitzakeelako:

- **HttpOnly:** cookie-ak script maltzurren bidez lortu litezke, eta parametro honi esker, JavaScript-ek *document.cookie* bidez haiek eskuratzea eragozten du.
- **SameSite:** cookie-ak nahi ez ditugun beste lekuetako eskeretan bidaiatu ahal izango lukete. Parametro honen bitartez gure domeinutik edo beste batetik bidali diren cookie-ak kontrolatzen ditugu.

Konpontzeko, cookie-ak konfiguratu ditugu. Lehen geneukan PHP bertsioa 7.2.2 zen, eta ez zuen SameSite parametroa onartzen. Beraz, hartu dugun lehenengo pausoa **PHP 7.3** bertsioa ezartzea izan da. Horretarako proiektuan dagoen **Dockerfile** fitxategia eguneratu da.

```
FROM php:7.3-apache
```

HttpsOnly eta SameSite parametroak cookie-etan jartzeko, hurrengo aldaketak gehitu dira **CSRF.php** fitxategian (cookie-ak konfiguratzez arduratuko da, besteak beste) :

```
session_set_cookie_params([
    'lifetime' => 0,
    'path' => '/',
    'domain' => null,
    'secure' => false,
    'httponly' => true,
    'samesite' => 'Strict'
]);
```